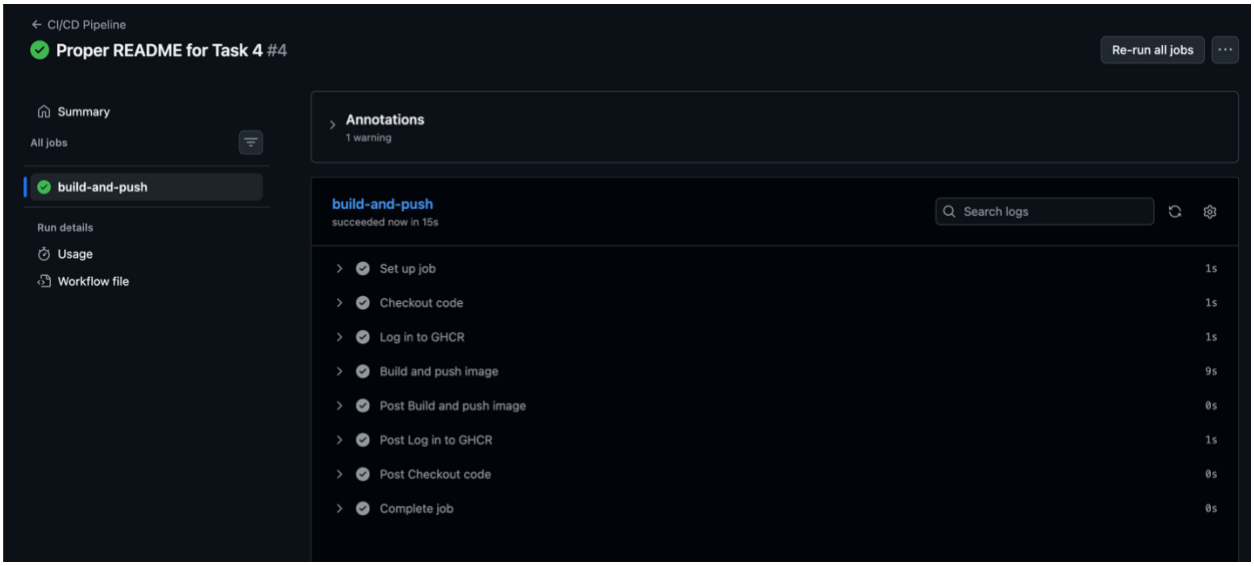
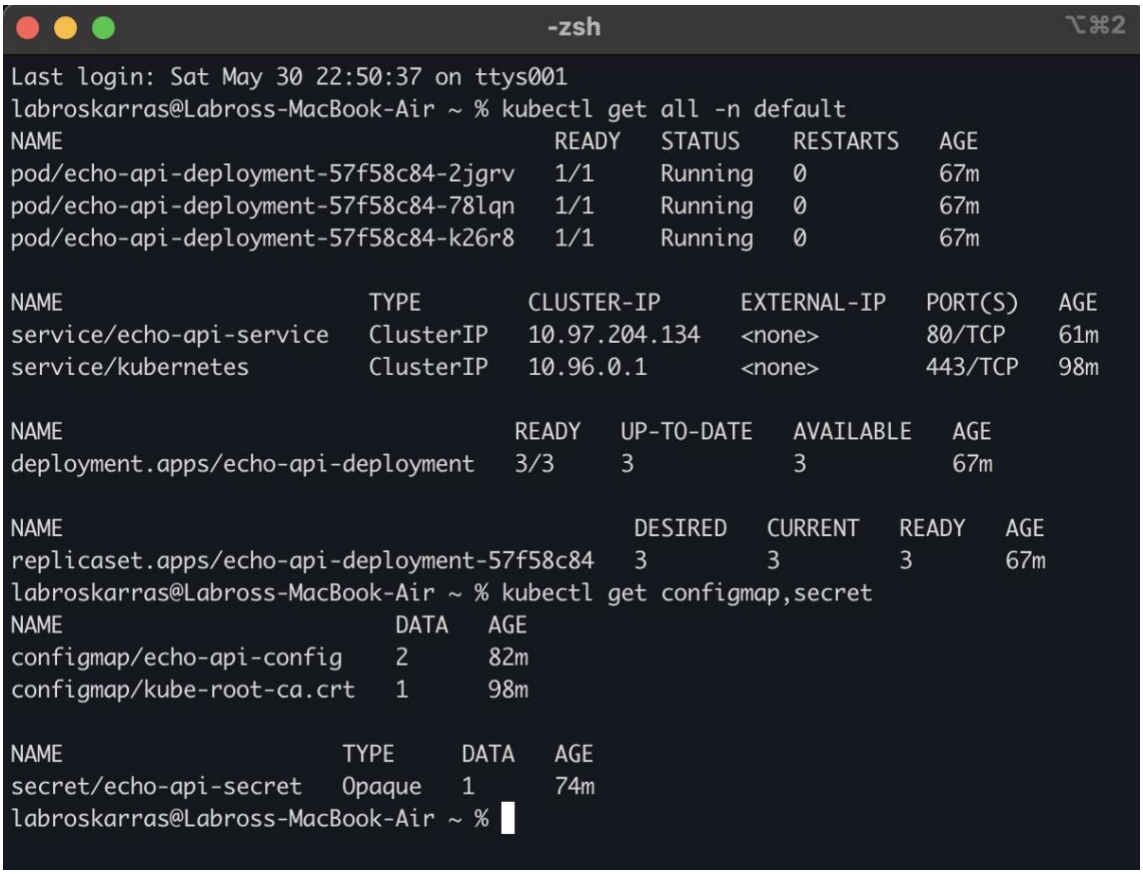
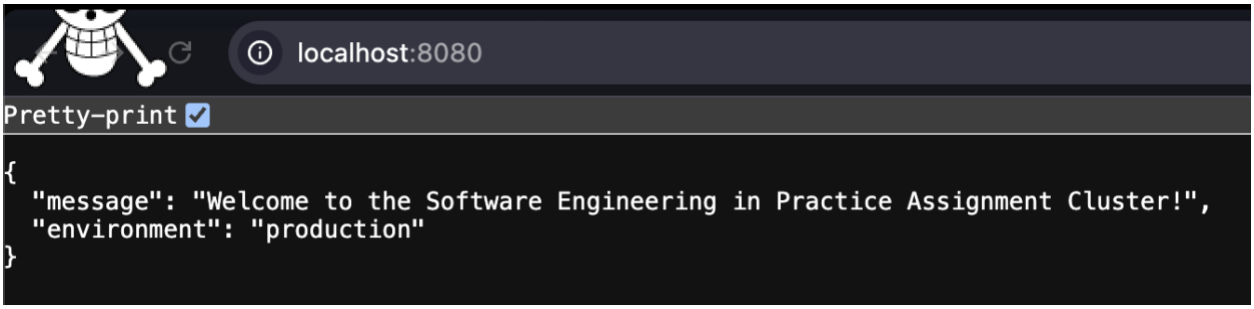
1. **GitHub Repository Link:**
   https://github.com/LabrosKarras/seip_assignment_1_2026
2. **CI/CD Proof:**

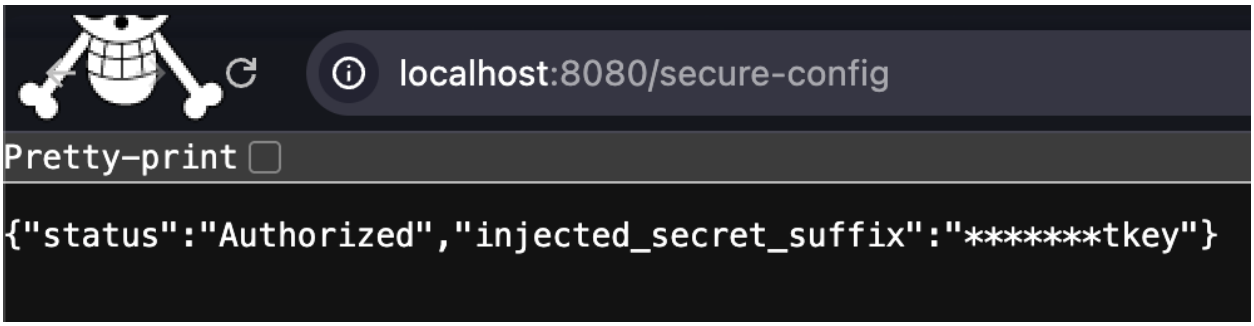


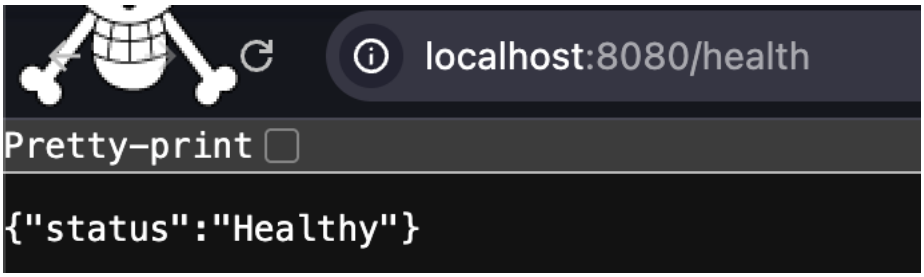
3. **Cluster State Proof:**



4. Application Verification Proof:







5. AI Reflection & Future Outlook:

a. **AI Integration:** Throughout this assignment, I primarily utilized Claude (Anthropic) and Google Gemini as my main AI assistance tools. I used AI to understand what Docker is and how it works internally (image layers, the build cache, the container lifecycle), what Kubernetes is and why it exists as an orchestration layer on top of Docker, and what Minikube's role is as a local single-node cluster for development purposes.

b. **Utility Analysis:** I also used AI for the core kubectl commands and understand what they were actually doing against the cluster rather than just copying them blindly. Apart from that, it helped understand Base64 encoding, why Kubernetes requires it for Secrets, and critically why the -n flag in echo -n is essential to avoid encoding a hidden newline character into the secret value. AI was also highly effective for syntax debugging of YAML files, where indentation errors are common and difficult to spot manually, and for explaining the exact mechanics of how environment variables flow from a ConfigMap or Secret into a running container's process environment. Finally, I used it’s assistance for formatting and

structuring the README.md file, including proper Markdown syntax for tables, code blocks, and headers.

c. **Friction Points:** The most significant friction point encountered was the GHCR image tag rejection due to uppercase characters. My GitHub username contains uppercase letters (LabrosKarras), and the GitHub Container Registry requires all repository names to be strictly lowercase. The AI did not proactively warn about this constraint when writing the initial workflow file, and it only surfaced as a runtime error after the first pipeline run failed. The fix was hardcoding my username with lowercase into the repo name, instead of using ${{ github.actor }}.

d. **Future Architectural Outlook:** If given additional time, I would establish a GitOps workflow with ArgoCD. Currently the CI/CD pipeline pushes an image but Kubernetes does not automatically detect and deploy the new version. ArgoCD would continuously monitor the Git repository and automatically synchronize the cluster state to match whatever is declared in the manifests, creating a true GitOps loop where Git is the single source of truth for the entire system state.